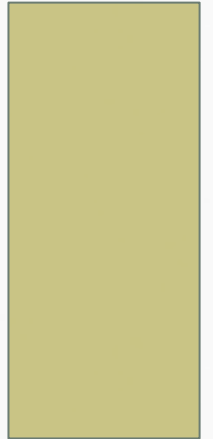


DATASTRUCTURES & ALGORITHM

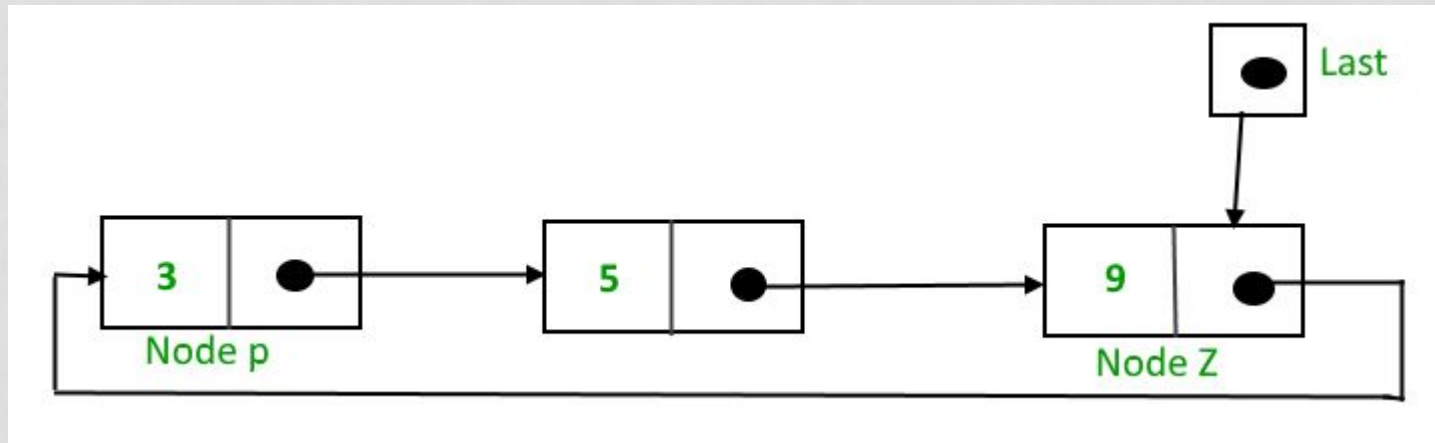
LECTURE 07
DATA ABSTRACTION & PROBLEM SOLVING WITH C++



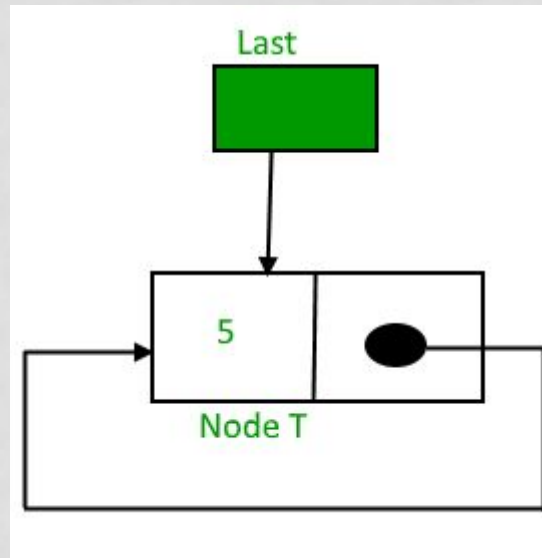
LINKED LIST APPLICATIONS

- Student Management System
 - Insert a new student In sequence w.r.t Roll no
 - Delete a student anywhere from list
 - Edit a student
- Audio Player
 - Create a play List
 - Play next Track
 - Delete a track from play list

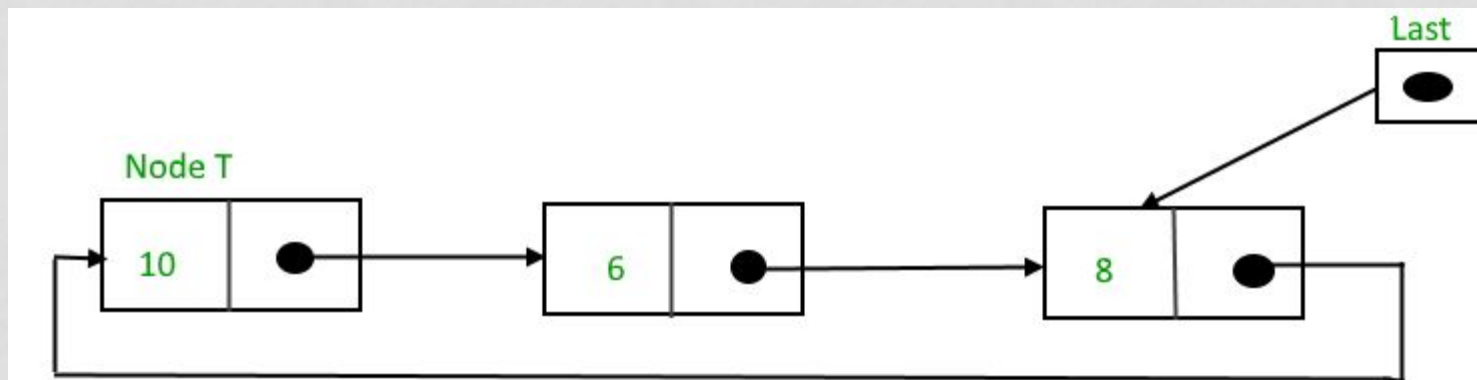
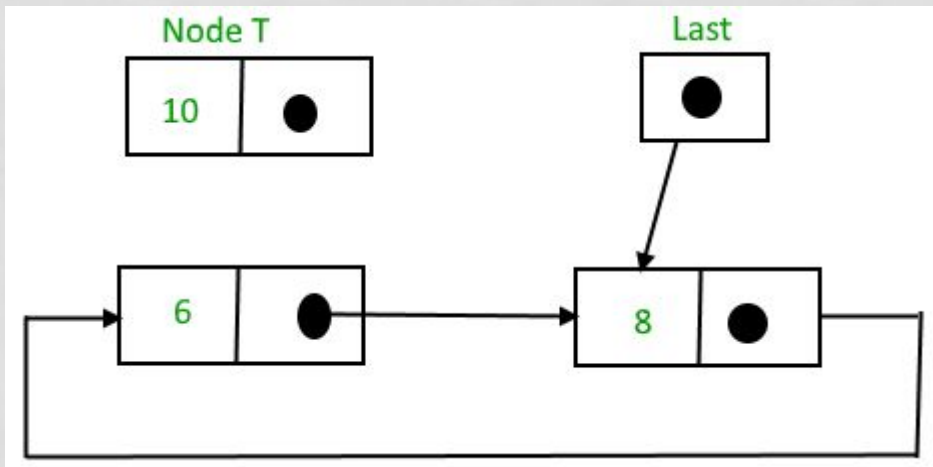
CIRCULAR LINKED LIST



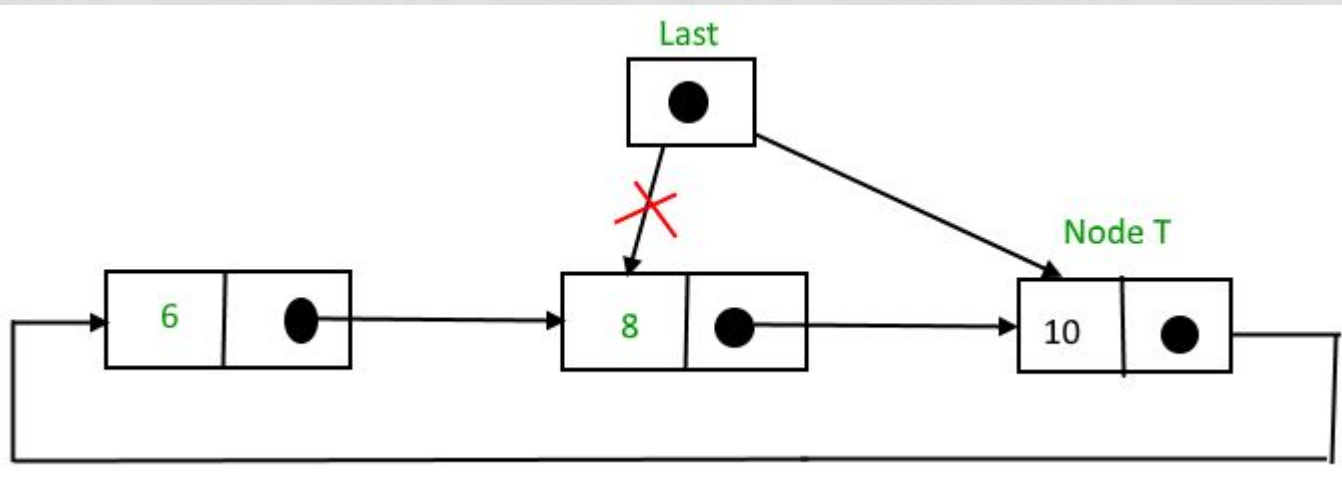
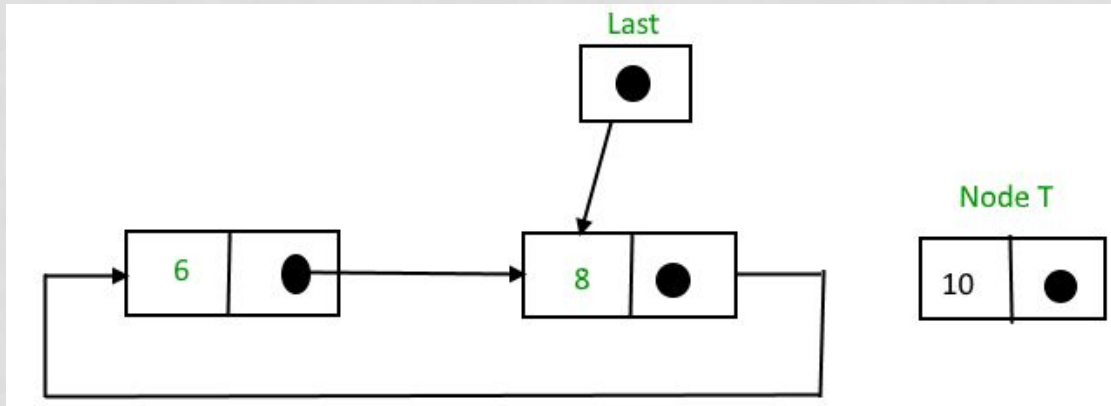
INSERTION IN EMPTY LIST



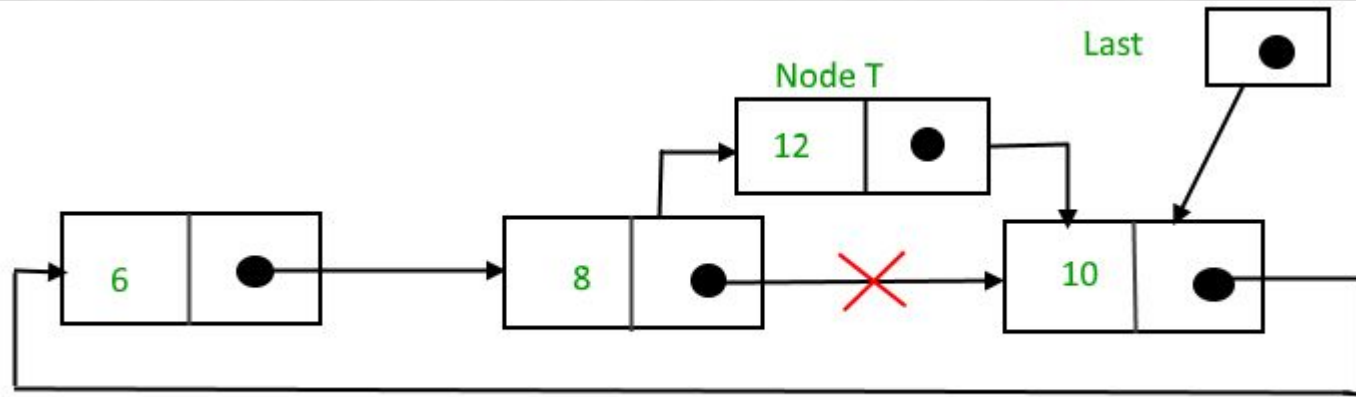
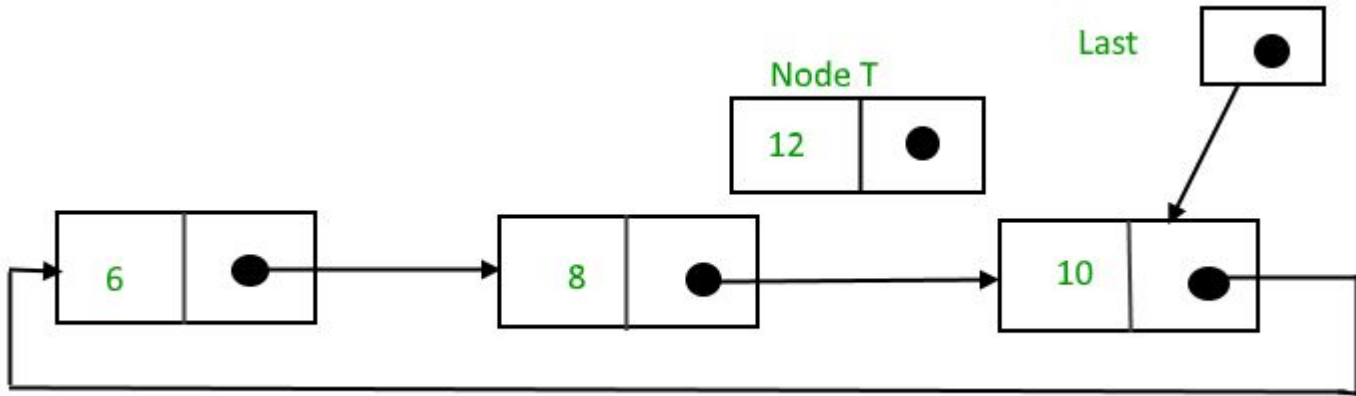
INSERT AT START OF NON EMPTY LIST



INSERT AT END OF NON EMPTY LIST



INSERT AT ANY POSITION

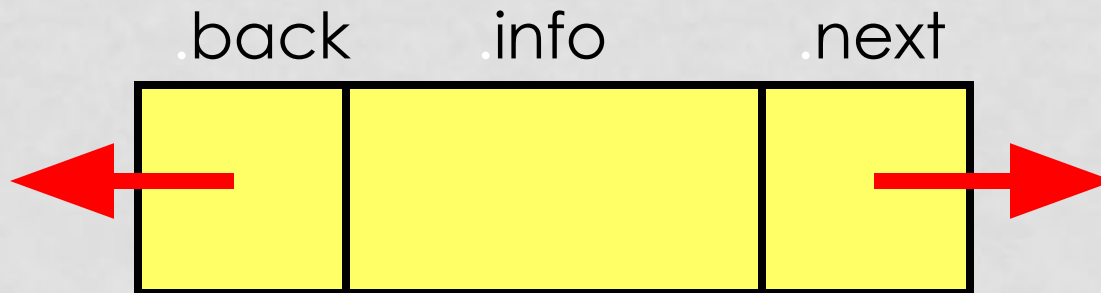


```
class CircularLinkedList
```

```
{  
CircularLinkedList();  
void insertAtStart(int c);  
void deleteAtStart();  
void deleteAtAnyPos();  
void insertAtEnd(int c);  
void deleteAtEnd();  
void insertAtAnyPos();  
void traverseForward();  
void traverseBackward();  
Bool isEmpty();  
~CircularLinkedList();  
CircularLinkedList(const CircularLinkedList& h);  
};
```

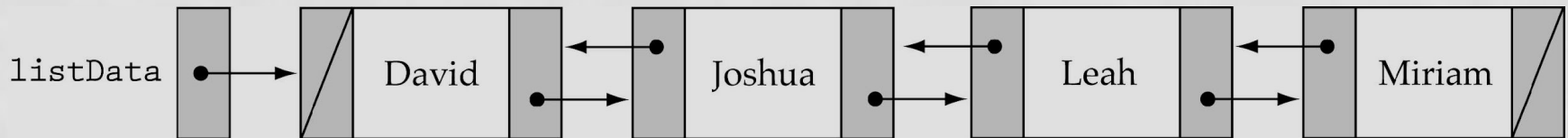

NODE DATA

- info: the user's data
- next, back: the address of the next and previous node in the list



NODE DATA (CONT.)

```
template<class ItemType>
struct Node {
    ItemType info;
    Node *next;
    Node *back;
};
```



DLLS COMPARED TO SLLS

- Advantages:

- Can be traversed in either direction (may be essential for some programs)
- Some operations, such as deletion and inserting before a node, become easier

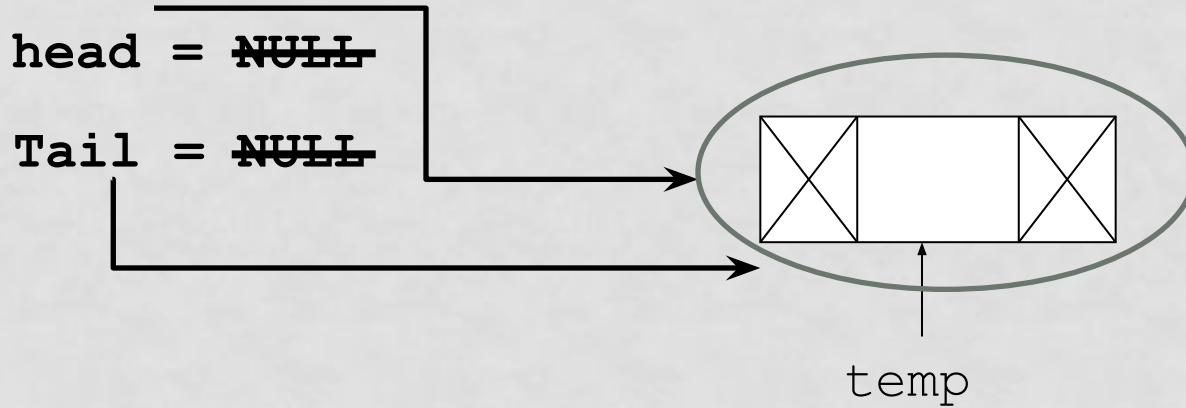
- Disadvantages:

- Requires more space
- List manipulations are slower (because more links must be changed)
- Greater chance of having bugs (because more links must be manipulated)

INSERTION

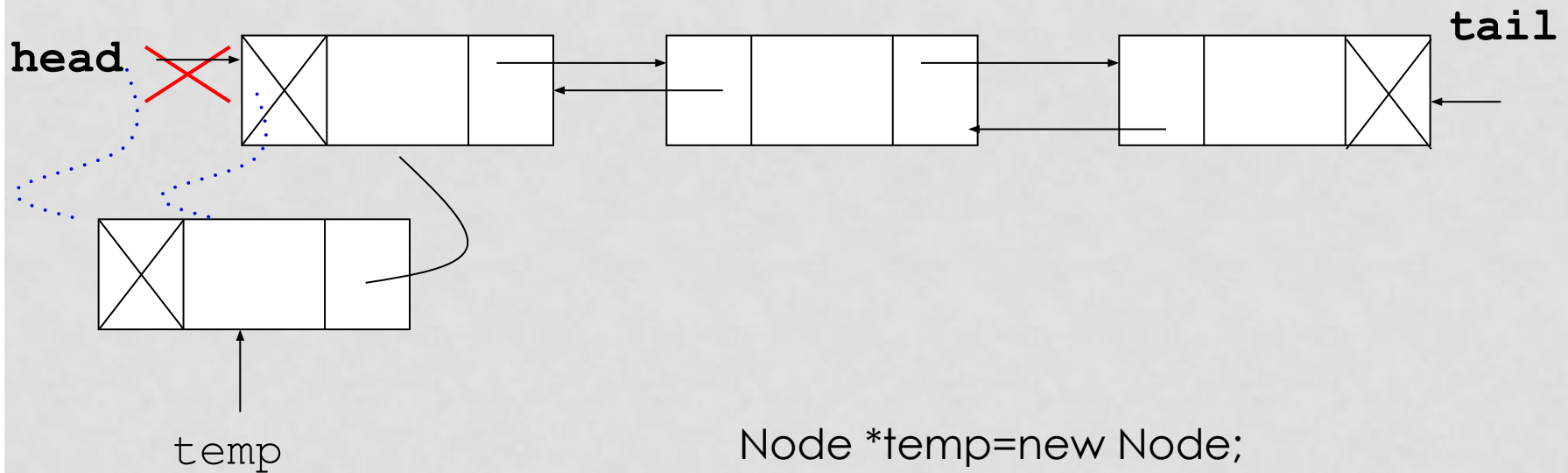
- Insert in an empty Linked List
- Insert at beginning of non empty list
- Insert at end of non empty list
- Insert somewhere in non empty list

INSERT IN EMPTY LIST



```
Node* temp= new Node;  
temp->next=NULL;  
temp->prev=NULL;  
head=temp;  
Tail=temp;
```

INSERT AT START OF NON EMPTY LIST



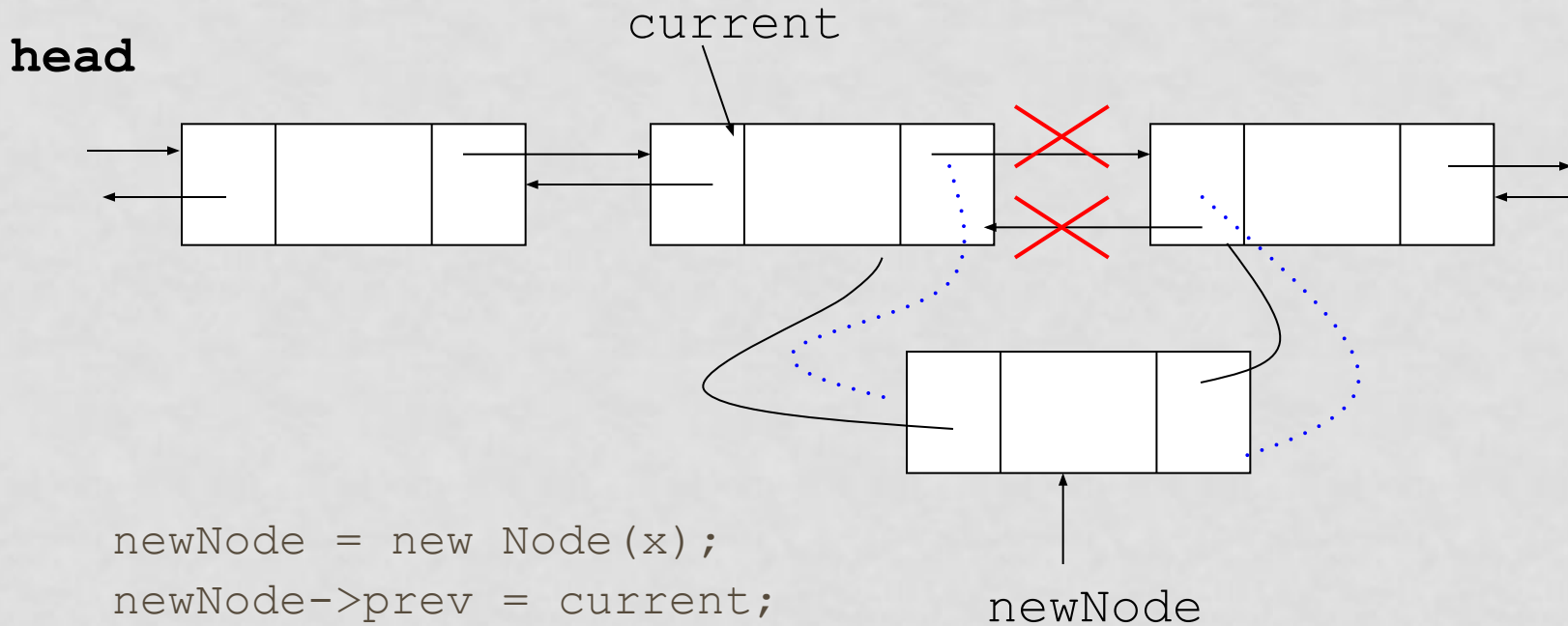
```
Node *temp=new Node;
```

```
Temp->next=head;
```

```
Head->prev=temp;
```

```
Head=temp;    Temp->prev=NULL;
```

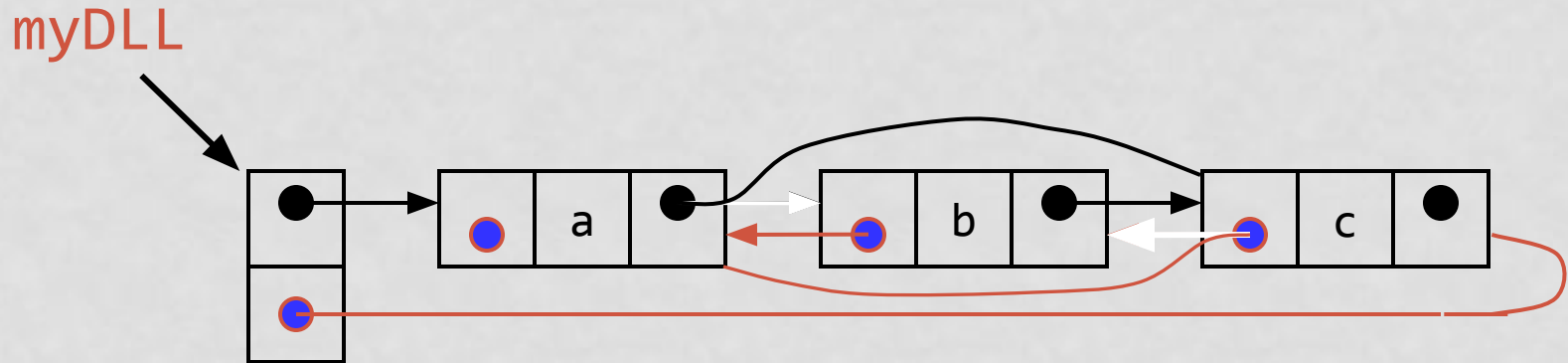
INSERTION SOMEWHERE



```
newNode = new Node(x);  
newNode->prev = current;  
newNode->next = current->next;  
current->next = newNode;  
newNode->next->prev = newNode;
```

DELETING A NODE FROM A DLL

- Node deletion from a DLL involves changing *two* links
- In this example, we will delete node b



- We don't have to do anything about the links in node b
- Garbage collection will take care of deleted nodes
- Deletion of the first node or the last node is a special case


```
class DoublyLinkedList
```

```
{  
DoublyLinkedList();  
void insertAtStart(int c);  
void deleteAtStart();  
void deleteAtAnyPos();  
void insertAtEnd(int c);  
void deleteAtEnd();  
void insertAtAnyPos();  
void traverseForward();  
void traverseBackward();  
Bool isEmpty();  
~DoublyLinkedList();  
DoublyLinkedList(const DoublyLinkedList & h);  
};
```

CLASS TASK

Design DoublyLinkedList ADT

Default constructor

Insert (at start, at end, at any pos)

Delete (at start, at end, at any pos)

Traverse (Forward, Backward)

Destructor

Copy constructor